Professional Guide: Integrating a Machine Learning Model with FastAPI and Oracle APEX

xAI

July 30, 2025

Contents

1 Introduction **2**

2 FastAPI Backend Setup **2**
2.1 Code Structure 2
2.2 Starting the FastAPI Service 2
2.3 Best Practices for FastAPI 3

3 Oracle APEX Integration **3**
3.1 APEX Page Setup 3
3.2 Dynamic Action for API Call 3
3.3 Displaying Results 4

4 Deployment Considerations **4**

5 Conclusion **4**

1 Introduction

This document provides a comprehensive guide for integrating a machine learning (ML) model into an Oracle APEX application using a FastAPI-based API. The solution leverages FastAPI for serving ML predictions and Oracle APEX for creating a user interface that interacts with the API. The guide covers the API setup, APEX integration, and best practices for deployment.

2 FastAPI Backend Setup

2.1 Code Structure

The FastAPI application serves as the backend, exposing an endpoint to deliver ML model predictions. Below is the core implementation.

```
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel
import joblib
import numpy as np

class InputData(BaseModel):
    num_steps: int

app = FastAPI(title="Forecast API")

MODEL_PATH = "./nb_ds_model.pkl"
with open(MODEL_PATH, "rb") as f:
    model_fit = joblib.load(f)

@app.post("/predict")
def predict(data: InputData):
    if data.num_steps <= 0:
        raise HTTPException(status_code=400, detail="num_steps must be positive")
    try:
        preds = model_fit.forecast(steps=data.num_steps)
        return {"prediction": np.asarray(preds).tolist()}
    except Exception as e:
        raise HTTPException(status_code=500, detail=f"Prediction error: {str(e)}")
```

2.2 Starting the FastAPI Service

To run the FastAPI service, install the required dependencies and start the server.

- **Install Dependencies:**

```
pip install fastapi uvicorn joblib numpy
```

- **Start the Server:**

```
uvicorn main:app --host 0.0.0.0 --port 8080 --workers 1
```

- **Test the Endpoint:**

```
curl -X POST "http://localhost:8080/predict" \
  -H "Content-Type: application/json" \
  -d '{"num_steps": 5}'
```

Expected response: `{"prediction": [12.3, 45.6, 78.9, 10.1, 11.2]}`

2.3 Best Practices for FastAPI

- **Serialization:** Convert `numpy.ndarray` to a list using `np.asarray(preds).tolist()` for JSON compatibility.
- **Validation:** Ensure `num_steps > 0` and handle model exceptions with `HTTPException`.
- **Security:** For production, use a reverse proxy (e.g., Nginx) with HTTPS, configure systemd for service management, and set reasonable timeouts.
- **Network Access:** Open port 8080 and allow the Oracle database/ORDS server IP to access the API. No CORS is needed if called from PL/SQL.

3 Oracle APEX Integration

3.1 APEX Page Setup

Create a page in Oracle APEX (e.g., Page 1) with the following components:

- **P1_INPUT:** Number Field for entering the number of prediction steps.
- **P1_OUTPUT:** Textarea or Display Only item to show the API response.
- **Button:** A "Predict" button to trigger the API call.

3.2 Dynamic Action for API Call

Add a Dynamic Action on the "Predict" button with the following settings:

- **Action:** Execute Server-side Code
- **Items to Submit:** `P1_INPUT`
- **Items to Return:** `P1_OUTPUT`

Use the following PL/SQL code to call the FastAPI endpoint and process the response:

```
DECLARE
  l_response_clob  CLOB;
  l_num_steps      NUMBER := :P1_INPUT;
  l_pretty_clob    CLOB;
BEGIN
  l_response_clob := APEX_WEB_SERVICE.make_rest_request(
    p_url         => 'http://<host>:8080/predict',
    p_http_method => 'POST',
    p_parm_name   => apex_util.string_to_table('Content-Type'),
    p_parm_value  => apex_util.string_to_table('application/json'),
    p_body        => '{"num_steps": ' || TO_CHAR(l_num_steps) || '}'
  );

  APEX_JSON.PARSE(l_response_clob);

```

```
  SELECT JSON_SERIALIZE(l_response_clob RETURNING CLOB PRETTY)
    INTO l_pretty_clob
    FROM dual;

  :P1_OUTPUT := APEX_STRING.CLOB_TO_VARCHAR2(l_pretty_clob);

EXCEPTION
  WHEN OTHERS THEN
    :P1_OUTPUT := 'Error: ' || SQLERRM;
END;
```

3.3 Displaying Results

- **Raw JSON:** Use a Textarea or Display Only item with `<pre>` tags to show the formatted JSON response.
- **Selective Values:** Parse the JSON in PL/SQL using `APEX_JSON` to extract specific prediction values and display them in a report or Display Only item.
- **Optional Refresh:** Add a True Action to refresh a report or region if predictions are stored in a collection or table.

4 Deployment Considerations

- **FastAPI Deployment:** Use a production-grade server (e.g., Gunicorn with Uvicorn workers) behind an Nginx reverse proxy. Configure HTTPS and monitor server performance.
- **APEX Security:** Ensure the APEX application restricts access to authorized users and validates inputs before API calls.
- **Error Handling:** Implement robust error handling in both FastAPI and APEX to provide meaningful feedback to users.
- **Scalability:** Monitor API usage and consider load balancing if the ML model serves multiple clients.

5 Conclusion

This guide outlines a robust approach to integrating an ML model with Oracle APEX using a FastAPI backend. By following the provided code and best practices, developers can create a seamless, production-ready solution for delivering ML predictions within an APEX application.